Implementing a Tree in C++

A complete guide to designing and writing a generic tree class in modern C++ (C++20). It covers the ideas behind a tree, the class-design practices a good C++ container follows, and every language and standard-library feature the final code uses.

Rule of this document: nothing appears in the code unless Part 1 has explained it first. Every concept has an id (`C1` , `C2` , ...). When the design (Part 2) or the code (Part 4) uses something, it cites that id so you can jump back.

Contents

- Part 0 — What we are building
- Part 1 — Required concepts
  - A. Files, names and the build
  - B. Classes and objects
  - C. Memory and ownership
  - D. Copy and move
  - E. Templates and concepts
  - F. Standard containers and utilities
  - G. Functions as values
  - H. Iterators and ranges
  - I. Errors and correctness
- Part 2 — Designing the tree
- Part 3 — Class-design checklist
- Part 4 — The implementation
- Part 5 — Tests
- Part 6 — Variants and when to use the standard library instead
- Part 7 — Exercises
- Appendix — Concept → where it is used

Part 0 — What we are building

A **rooted, ordered, N-ary tree** holding values of any type `T` :

- *rooted*: one node, the root, has no parent;
- *ordered*: a node's children have a fixed left-to-right order;
- *N-ary*: a node may have any number of children (0, 1, 2, ...).

A binary tree and a binary search tree are special cases; Part 6 shows how the design changes for them.

The finished class, `dsa::Tree<T>`, offers:

Group	Operations
Build	`Tree()`, `Tree(rootValue)`, `emplaceRoot`, `addChild`, `emplaceChild`
Restructure	`detach` (cut out a subtree as its own tree), `attach`, `erase`, `clear`
Query	`size`, `empty`, `root`, `height`, `depth`, `contains`, `find`, `findIf`
Traverse	`forEachPreOrder`, `forEachPostOrder`, `forEachLevel`, range-for / iterators
Value semantics	deep copy, cheap move, `swap`, `==`

It passes a test suite (Part 5), built with `-Wall -Wextra -Wpedantic -Werror` and run under AddressSanitizer and UndefinedBehaviorSanitizer. A release build handles a chain **one million levels deep**. A naive recursive destructor crashes (segfaults) at that depth. Part 2.8 explains why.

Build commands used throughout (GCC 13; Clang 16+ also works):

```
# debug: assertions on, sanitizers catch memory errors and undefined behaviour
g++ -std=c++20 -Wall -Wextra -Wpedantic -Werror -g -fsanitize=address,undefined test.cpp -o test_debug
./test_debug

# release: optimised, assertions off (-DNDEBUG), deep-tree stress test
g++ -std=c++20 -O2 -DNDEBUG -Wall -Wextra -Wpedantic -Werror test.cpp -o test_release
./test_release 1000000
```

Part 1 — Required concepts

A. Files, names and the build

C1. Headers, `#include`, and `#pragma once`

C++ compiles each `.cpp` file separately (a *translation unit*). A header (`.hpp`) holds declarations that several `.cpp` files share. `#include "Tree.hpp"` pastes the header's text in place. `#include <vector>` does the same for a standard-library header; angle brackets mean "search the system/library paths", quotes mean "search next to this file first".

If one translation unit included the same header twice, its classes would be defined twice, which is a compile error. The fix is a guard. You have two good options:

Option	Code	Notes
`#pragma once`	`#pragma once` at the top	Not in the standard, but every major compiler supports it. Short, and you can't make a typo in a macro name. The existing `Trie.hpp` in this repo uses it.
Include guard	`#ifndef DSA_TREE_HPP` / `#define DSA_TREE_HPP` / … / `#endif`	100% standard. The macro name must be unique across the project.
(C++20 modules)	`export module dsa.tree;`	The long-term replacement for headers. Build-system support is still uneven, so this document doesn't use them.

Why the tree lives entirely in a header: it is a template (C8), and the compiler must see a template's full definition wherever it is used.

Include what you use: each file includes the headers for exactly the names it uses, and doesn't rely on another header happening to pull them in. The table in Part 4.1 maps every include to the names it provides.

C2. Namespaces

A namespace groups names so they don't collide with names elsewhere (`dsa::Tree` vs someone else's `Tree`). The standard library lives in `std` .

```
namespace dsa
{
class Tree { /* ... */ };
} // namespace dsa      <- comment the closing brace; it's far from the opening one

dsa::Tree t;           // fully qualified use
```

Good practice:

- Never write `using namespace std;` in a header. It leaks into every file that includes the header, and can silently change which overload gets picked.
- Inside a `.cpp` or a function body, `using std::swap;` (a *using declaration* for one name) is fine and sometimes needed (see C15).

C3. Naming conventions

The language doesn't care about naming style; readers do. Pick one style and apply it consistently. Common member-variable styles:

Style	Example	Seen in
`m` prefix	`mRoot`, `mChildren`	this repo (`Trie.hpp`), many game/engine codebases
`m_` prefix	`m_root`	MFC/Qt-influenced code
trailing underscore	`root_`	Google style, Chromium

This document follows the repo: `mCamelCase` for data members, `camelCase` for functions, `PascalCase` for types and template parameters. Standard-like type aliases (`value_type`, `size_type`, `iterator`) stay in snake_case because generic code (and the standard library) look for those exact names (C27).

C4. `std::size_t`, `std::ptrdiff_t`, and digit separators

- `std::size_t` (from `<cstddef>`) is the unsigned integer type used for sizes and indices. `vector::size()` returns it. Using `int` for sizes causes signed/unsigned comparison warnings and overflows past about 2 billion.
- `std::ptrdiff_t` is the signed type for the distance between two iterators/pointers. Iterators must declare it as `difference_type` (C27).
- `1'000'000` is the number one million; the `'` separator (C++14) only helps readability.

B. Classes and objects

C5. `class` vs `struct`, access specifiers, encapsulation and invariants

`class` and `struct` are the same thing except for the default access: `struct` members are `public` by default, `class` members are `private`.

- `public:` anyone can use it.
- `private:` only the class's own member functions (and friends, C11) can use it.
- `protected:` like private, plus derived classes. A container that isn't meant to be a base class doesn't need it.

Convention: use `struct` for passive bundles of data with no rules (like `TrieNode` in this repo). Use `class` when the data has an **invariant**, a rule that must always hold. Examples: "`mSize` equals the number of nodes", or "each child's `mParent` points back at its parent". If data members were public, any user could break the rule. Making them private and changing them only through member functions is **encapsulation**. It turns "the invariant holds" into something you can check by reading one class. Write the invariants down in a comment at the top of the class; `Tree` does.

C6. Constructors, member initializer lists, default member initializers, `explicit`

A constructor runs when an object is created and must leave it satisfying its invariants.

```
class Node
{
public:
    Node(Node* parent, int value)
        : mValue(value), mParent(parent)   // member initializer list
    {
    }
private:
    int mValue;
    Node* mParent{nullptr};                // default member initializer
};
```

- **Member initializer list** (`: mValue(value), …`) *initializes* members. Assigning them inside the body `{ mValue = value; }` would first default-construct each member, then overwrite it. That is slower, and impossible for members that can't be default-constructed or assigned (references, `const` members). Members are always initialized in **declaration order**, whatever order the list uses. Keep both orders the same; `-Wall` warns when they differ.
- **Default member initializer** (`Node* mParent{nullptr};`) gives a member a value whenever a constructor doesn't mention it, so no constructor can forget it. Prefer this for simple defaults.
- **`explicit`** on a one-argument constructor stops implicit conversions. Without it, `Tree<int> t = 5;` or passing `5` to a function expecting a `Tree<int>` would silently build a tree. Rule: make single-argument constructors `explicit` unless the conversion is genuinely natural (like `std::string` from `"text"`).

C7. **`= default`** and **`= delete`**

- `Tree() = default;` asks the compiler to generate the default constructor. It default-initializes each member, using the default member initializers from C6. Prefer this to writing an empty `{}` body: it keeps the class simple and can keep it trivially constructible.
- `Node(const Node&) = delete;` forbids an operation. Code that tries to copy a `Node` fails to compile with a clear message. We delete `Node`'s copy operations because a copied node would have a parent that doesn't own it, which breaks an invariant.

C8. Member functions: **`const`**, overloading on **`const`**, **`noexcept`**, **`[[nodiscard]]`**, **`static`**

`const` **member functions.** A `const` after the parameter list promises the function doesn't change the object: `std::size_t size() const;`. You can call only `const` member functions on a `const` object or through a `const&`. **Good practice: mark every function that doesn't modify observable state** **`const`**. This is *const-correctness*. It lets users pass `const Tree&` safely, and the compiler enforces it.

Overloading on **`const`**. You can provide two versions of a function, one for mutable objects and one for `const` ones:

```
T& value() noexcept { return mValue; }             // tree is mutable -> you may change the value
const T& value() const noexcept { return mValue; } // tree is const   -> read-only
```

The compiler picks the version based on the constness of the object. The standard containers do the same (`vector::operator[]`, `begin()`, …).

Avoiding duplication between the two versions. When the bodies are more than one line, you have several options:

Option	How	Trade-off
Duplicate	write both bodies	fine for one-liners (our getters)
Static template helper	a private `template <typename NodePtr> static … impl(NodePtr p)`, called with `Node*` or `const Node*`	one body, no casts. **Used for our traversals.**
`const_cast`	non-const version calls the const one and casts the result	one body, but a cast you must get right
C++23 "deducing this"	`template <class Self> auto& value(this Self& self)`	cleanest, needs C++23

`noexcept` declares that a function never throws. Two reasons to use it:

1. Documentation and optimisation: callers don't need exception-handling paths for it.
2. **It changes behaviour in the standard library.** `std::vector` moves elements when it grows only if their move constructor is `noexcept`; otherwise it copies them to keep its exception guarantees (C19, C30). So move constructors, move assignment, `swap` and destructors should be `noexcept`.

If a `noexcept` function does throw, the program calls `std::terminate`. So mark a function `noexcept` only when you're sure it won't throw, or when crashing is the right response to a throw (see `clear()` in Part 4).

`[[nodiscard]]` is an attribute that makes the compiler warn when a caller ignores the return value. Put it on functions whose only point is the value they return (`size()`, `empty()`, `find()`). It catches the classic bug `v.empty();` written instead of `v.clear();`. It matters most on `detach()`: ignoring its result destroys the subtree you just cut out. Ignoring it on purpose needs an explicit `static_cast<void>(...)` (C10).

`static` member functions belong to the class, not to an object. They have no `this` and are called as `Tree<int>::depth(node)`. Use them for helpers that don't need a particular tree. For example, the depth of a node depends only on its parent pointers.

C9. Nested classes and type aliases (`using`)

A class defined inside another (`class Tree { class Node { … }; };`) is a **nested class**. Its full name is `Tree<T>::Node`. Nesting says "this type exists only as part of a tree" and keeps the namespace tidy. The standard library does the same with `std::map<K,V>::iterator`.

A **type alias** gives a type a new name:

```
using size_type = std::size_t;
using iterator  = PreOrderIterator<false>;
```

`using X = Y;` is the modern form of `typedef Y X;` and reads left to right. Containers publish aliases such as `value_type`, `size_type` and `iterator` so generic code can ask `typename Container::value_type` without knowing the container.

A nested class is a member of its enclosing class, so it can access the enclosing class's private members.

C10. Casts: `static_cast`, and why to avoid the others

`static_cast<Target>(expr)` performs a conversion the compiler can check at compile time. We use it twice:

- `static_cast<const Node*>(ptr)` adds `const` to a pointer, to pick the `const Node*` version of a template helper (C8).
- `static_cast<void>(detach(node))` discards a `[[nodiscard]]` result on purpose and says so in the code.

The other casts: `const_cast` removes `const` (rarely correct), `reinterpret_cast` reinterprets raw bits (almost never needed in a container), and C-style `(T)x` can silently do any of these. Avoid all three here.

C11. `friend`, and "hidden friends"

A class can grant another class or function access to its private members:

```
class Node
{
    friend class Tree;       // Tree's member functions may touch Node's privates
};
```

`Node`'s constructor and data are private, so users can't create nodes or rewire parents, but `Tree` can. Friendship is deliberate, narrow encapsulation, not a hole in it.

A **hidden friend** is a non-member function defined *inside* the class with `friend`:

```
friend void swap(Tree& a, Tree& b) noexcept { /* ... */ }
friend bool operator==(const Tree& a, const Tree& b) { /* ... */ }
```

It is a free function (you call `swap(a, b)`, not `a.swap(b)`), but it can be found only through **argument-dependent lookup (ADL)**. ADL means the compiler looks inside the classes and namespaces of the argument types, so the function is found only when a `Tree` is an argument.

This is the recommended way to write operators and `swap` for a class: it doesn't clutter overload resolution for unrelated types, and it can use the class's private members.

C12. Operator overloading

You give an operator meaning for your type by defining a function named `operator<symbol>`. The ones we need:

Operator	Signature in our code	Meaning
`*` (unary)	`reference operator*() const`	iterator: the current element
`->`	`pointer operator->() const`	iterator: `it->member`
prefix `++`	`Iterator& operator++()`	advance, return self
postfix `++`	`Iterator operator++(int)`	the dummy `int` marks postfix; returns the *old* value (a copy)
`==`	`friend bool operator==(const X&, const X&)`	equality
`=`	`Tree& operator=(Tree other)`	assignment (C17)

In **C++20**, if you define `==`, the compiler rewrites `a != b` as `!(a == b)` for you. You don't write `!=` any more. (Similarly, `operator<=>`, the "spaceship", generates `<`, `<=`, `>`, `>=`.) If a class's `==` just compares every member, `bool operator==(const X&) const = default;` generates it. Ours can't be defaulted: it compares tree *structure*, not the pointer values.

Good practice: an overloaded operator should do what its built-in counterpart does (don't make `+` subtract). Prefix `++` should return a reference, and postfix a copy.

C. Memory and ownership

C13. Object lifetime, the stack and the heap, RAII, destructors

- **Automatic (stack) objects** such as `int x;` or `std::vector<int> v;` inside a function die automatically at the closing `}`, in reverse order of creation.
- **Dynamic (heap) objects** are created with `new` and live until someone calls `delete`. Forgetting the `delete` leaks memory; calling it twice, or using the object afterwards, is undefined behaviour.

A **destructor** `~Tree()` runs when an object dies. **RAII** ("Resource Acquisition Is Initialization") is *the* core C++ idiom. An object acquires a resource (memory, a file, a lock) in its constructor and releases it in its destructor. The resource's lifetime is then tied to the object's scope, so it is released on every path out of the scope: normal return, early `return`, or an exception. You never write `delete` by hand; a class such as `std::unique_ptr` (C14) does it for you.

The call stack is small, typically **8 MB on Linux**. Every active function call uses some of it. Deep recursion (a function calling itself a million times) overflows it and crashes. This matters for trees; see Part 2.8.

C14. Raw pointers, references, and smart pointers

References (`T&`) are aliases for an existing object. A reference can't be null and can't be reseated to refer to something else. Use one when "there is always exactly one object here".

Raw pointers (`T*`) may be `nullptr` and may be reseated. **Modern rule: a raw pointer never owns.** It *observes* an object that something else keeps alive. `nullptr` (C++11) is the typed null pointer; don't use `0` or `NULL` .

Smart pointers (`<memory>`) are RAII objects that own heap memory:

Type	Ownership	Cost	Use when
`std::unique_ptr<T>`	exactly one owner; move-only (C16)	same size as a raw pointer, no overhead	**the default**. A parent owns its children.
`std::shared_ptr<T>`	shared; a reference count deletes the object when the last owner goes	two pointers wide, atomic count updates	ownership really is shared and has no natural single owner
`std::weak_ptr<T>`	non-owning observer of a `shared_ptr` object; `lock()` tells you if it's still alive	as shared	break `shared_ptr` cycles (child → parent)

Operations we use on `unique_ptr` :

```
std::unique_ptr<Node> p(new Node(...));  // take ownership of a new object
Node* raw = p.get();                     // observe without taking ownership
Node& ref = *p;                          // dereference
std::unique_ptr<Node> q = std::move(p);  // transfer ownership; p becomes nullptr
p == nullptr;                            // test for "owns nothing"
                                         // at scope end: deletes whatever it owns
```

`std::make_unique<T>(args...)` is the usual way to create one. It is shorter, never leaves you holding a naked `new` , and is exception-safe. **But it can only call a constructor it can access.** Our `Node` constructor is private, and `make_unique` is not a friend, so `Tree` writes `std::unique_ptr<Node>(new Node(...))` itself, inside one private helper. That's safe because the `new` expression is handed to the `unique_ptr` straight away. There is no gap where the node is unowned. (An alternative is the "passkey idiom": a public constructor that takes an argument of a private tag type that only `Tree` can create.)

D. Copy and move

C15. Value semantics and the special member functions

C++ types behave like values by default: `b = a` makes `b` an independent copy. For a container this means a **deep copy**: copying a tree copies every node, and changing the copy doesn't touch the original. That is how `std::vector` and `std::map` behave, and users expect the same from a tree.

The compiler can generate six **special member functions**: the default constructor, the destructor, the copy constructor `X(const X&)`, copy assignment `X& operator=(const X&)`, the move constructor `X(X&&)`, and move assignment `X& operator=(X&&)`.

Rule of zero / three / five:

- **Rule of zero** (preferred): if every member manages itself (vectors, strings, smart pointers), declare *none* of these. The generated versions are correct.
- **Rule of five:** if you need to write *any* of the destructor, copy operations or move operations, you almost certainly need to think about all five. (Before C++11 this was the "rule of three", with no moves.)

`Tree` applies the rule of five on purpose. Its only owning member is a `unique_ptr`, which can't be copied, so the generated copy constructor would be deleted, but we want deep copies. And the generated destructor would recurse down the tree (Part 2.8). *Rule-of-zero option:* if you accept a move-only tree and trees are never deeper than a few thousand levels, delete all five declarations and the class still works.

`swap` and `using std::swap;`. A cheap, `noexcept` `swap` is the building block for the assignment operator (C17). Inside our `swap`, we write `using std::swap; swap(a.mRoot, b.mRoot);`. This two-step idiom calls a type's own `swap` if ADL (C11) finds one, and falls back to `std::swap` otherwise.

C16. Move semantics: rvalue references, `std::move`, `std::exchange`

Copying a tree with a million nodes is expensive. Returning a tree from a function, or putting it into another one, shouldn't need a copy: the source is about to disappear anyway. **Moving** transfers the source's resources (here, the root pointer) and leaves the source valid but empty.

- An **rvalue reference** `Tree&&` binds to temporaries and to objects you've marked as expendable. A constructor taking `Tree&&` is the move constructor.
- **`std::move(x)`** (`<utility>`) moves nothing by itself. It's a cast that says "treat `x` as expendable", which makes the move overloads get picked. After `std::move(x)`, don't use `x` except to destroy it or assign it a new value. Standard types leave a moved-from object in a "valid but unspecified" state; our `Tree` guarantees *empty*.
- **`std::exchange(obj, newValue)`** (`<utility>`) sets `obj` to `newValue` and returns the old value in one expression. It's ideal in move constructors: `mSize(std::exchange(other.mSize, 0))` takes other's size and zeroes it.

Moving a `Tree` costs O(1) and can't throw, so it is `noexcept` (C8). Returning a local tree by value (`return result;`) moves it, or skips the move entirely (*copy elision* / RVO). **Return by value; don't return pointers to heap trees.**

C17. Copy-and-swap assignment

Assignment must handle self-assignment (`a = a`), free the old contents, and ideally leave `*this` untouched if copying throws. One idiom solves all three:

```
Tree& operator=(Tree other) noexcept   // 1. `other` is built by the copy OR move constructor
{
    swap(*this, other);                 // 2. swap our contents with it (noexcept)
    return *this;
}                                       // 3. `other` (holding our old contents) is destroyed
```

- Taking `other` **by value** means the caller's argument is copied (for an lvalue) or moved (for an rvalue) *before* the function body runs. If the copy throws, `*this` hasn't been touched yet. That's the strong guarantee (C30).
- This single operator serves as **both** copy and move assignment. Don't also declare `operator=(Tree&&)`, or calls become ambiguous.

Option: write separate copy and move assignment operators. This can be faster when `*this` already holds storage it could reuse (like `vector` reusing its capacity). It's not worth it for a tree, whose nodes can't be reused.

C18. Passing parameters: `const T&`, `T&&`, by-value "sink", and emplacing

How should `addChild` accept a value? Good options:

Option	Signature	Cost for an lvalue / an rvalue argument
Two overloads	`addChild(Node&, const T&)` and `addChild(Node&, T&&)`	1 copy / 1 move. Optimal, but two functions. **Used.**
By-value sink	`addChild(Node&, T value)` then `std::move(value)` inside	1 copy + 1 move / 2 moves. One function, slightly more work. **Used for the `Tree(T rootValue)` constructor**, which runs once.
Emplace (perfect forwarding, C22)	`emplaceChild(Node&, Args&&... args)`	builds `T` *directly inside the node* from constructor arguments: 0 copies, 0 moves

The standard containers offer both styles too: `push_back(value)` and `emplace_back(args...)`.

E. Templates and concepts

C19. Class templates and member templates

A **class template** is a pattern the compiler stamps out for each type you use:

```
template <typename T>        // T is a type parameter ("typename" and "class" mean the same here)
class Tree { T mValue; };

dsa::Tree<int> a;            // the compiler generates Tree<int>
dsa::Tree<std::string> b;    // ...and a separate Tree<std::string>
```

Only member functions that are actually *called* get compiled for a given `T`. So `Tree<T>` works for a `T` without `==`, as long as you don't call `find` on it. Concepts (C21) turn a misuse into a clear error message.

A **member template** is a template inside a class (or class template): `template <typename Pred> Node* findIf(Pred pred);` is a family of functions, one for each callable type.

`auto` and `decltype`. `auto x = expr;` lets the compiler deduce the variable's type from its initializer: `auto* tip = chain.root();` is a pointer to `Node`. `const auto&` binds a read-only reference without a copy. `auto` drops top-level `const` and `&`, so write `auto&` or `const auto&` when you want a reference. `decltype(expr)` names the exact type of an expression, keeping `&` and `const`. The tests use it to check that a const tree's iterator yields `const int&`. Inside a member function, `this` is a pointer to the current object and `*this` is the object itself.

Templates must be fully defined in headers (C1). Inside the class, the plain name `Tree` means `Tree<T>` (the *injected class name*).

C20. Non-type template parameters and `std::conditional_t`

A template parameter can also be a value: `template <bool IsConst> class PreOrderIterator`. `PreOrderIterator<true>` and `PreOrderIterator<false>` are different types generated from one definition.

`std::conditional_t<Cond, A, B>` (`<type_traits>`) is the type `A` if `Cond` is `true`, otherwise `B`, picked at compile time:

```
using NodePtr = std::conditional_t<IsConst, const Node*, Node*>;
```

That's how one iterator template produces both `iterator` (yields `T&`) and `const_iterator` (yields `const T&`) without duplicated code.

Other `<type_traits>` names used in the tests: `std::is_same_v<A, B>` (are two types identical?), `std::is_copy_constructible_v<X>` and `std::is_nothrow_move_constructible_v<X>`. The `_v` suffix gives the `bool` value directly.

C21. Concepts and `requires` (C++20)

A **concept** is a named compile-time predicate on types (`<concepts>`). It lets a template state its requirements up front instead of failing deep inside with pages of errors.

```cpp
// 1. Constrained template parameter: Pred must be callable with const T& and return something bool-like
template <std::predicate<const T&> Pred>
Node* findIf(Pred pred);

// 2. Trailing requires-clause: this member exists only if T supports ==
Node* find(const T& target) requires std::equality_comparable<T>;

// 3. In a static_assert, as a test (C29)
static_assert(std::forward_iterator<Tree<int>::iterator>);
```

Standard concepts we use:

Concept	Header	Meaning
`std::predicate<F, Args...>`	`<concepts>`	`F` is callable with `Args` and returns something usable as `bool`
`std::equality_comparable<T>`	`<concepts>`	`a == b` and `a != b` are valid for `T`
`std::forward_iterator<I>`	`<iterator>`	`I` satisfies every forward-iterator requirement (C27)
`std::ranges::forward_range<R>`	`<ranges>`	`begin(r)` / `end(r)` give forward iterators

Options for constraining templates, oldest to newest: nothing (errors show up late and are unreadable); SFINAE with `std::enable_if` (works but is cryptic); **concepts** (C++20, clear, **used**).

C22. Variadic templates, forwarding references, and `std::forward`

```cpp
template <typename... Args>                // a "parameter pack": zero or more types
Node& emplaceChild(Node& parent, Args&&... args)
{
    // ... T(std::forward<Args>(args)...) ...  // the "..." expands the pack
}
```

- `Args&&` where `Args` is a *deduced template parameter* isn't an rvalue reference. It's a **forwarding reference**, which binds to anything and remembers whether the argument was an lvalue or an rvalue.
- `std::forward<Args>(args)` (`<utility>`) passes each argument on *as it came in*: lvalues stay lvalues (copied), rvalues stay rvalues (moved). This combination is **perfect forwarding**, and it's how `vector::emplace_back` works.
- `std::move` vs `std::forward` : use `move` when you *know* you own something expendable, and `forward` only on forwarding references.

Example: `words.emplaceRoot(3, 'a')` forwards `3` and `'a'` to the `std::string(3, 'a')` constructor, building `"aaa"` directly inside the node.

F. Standard containers and utilities

C23. `std::vector`

A dynamic array (`<vector>`): contiguous memory, O(1) index access, and *amortised* O(1) append. It is **the default container**. Reach for another one only for a concrete reason.

Operations used:

Call	Effect
`push_back(x)` / `emplace_back(args...)`	append (copy/move `x`, or construct in place)
`back()` / `pop_back()`	last element / remove it (so a vector works as a stack)
`size()`, `empty()`	size / whether it's empty
`reserve(n)`	allocate room for `n` elements without changing `size()`
`at(i)`	bounds-checked access; throws `std::out_of_range` (`operator[]` is unchecked)
`erase(it)`	remove the element at iterator `it`; later elements shift left
`rbegin()`, `rend()`	reverse iterators, for walking backwards

Iterator/reference invalidation, the vector's big trap. When `push_back` needs more capacity, the vector allocates a new buffer, moves the elements across and frees the old one. **Every pointer, reference and iterator into the vector then dangles.** Two consequences for us:

1. In `postOrder` we hold `auto& [current, nextChild] = stack.back()` and then `emplace_back` onto the same stack. We read everything we need *before* the `emplace_back` and don't touch those references afterwards.
2. Our nodes live in *separately allocated* heap blocks (via `unique_ptr`). The children vector holds pointers, so growing it moves pointers, not nodes. **`Node&` handles given to users stay valid** whatever happens to other nodes. A `std::vector<Node>` arena doesn't have this property (Part 6.3).

Why `reserve(n + 1)` before `push_back`: after the `reserve` succeeds, the next `push_back` is guaranteed not to allocate, so it can't throw. We do the only step that can fail *first*, before changing anything. That's how `emplaceChild` and `attach` get the strong guarantee (C30).

C24. `std::queue`, `std::pair`, and structured bindings

- **`std::queue<T>`** (`<queue>`) is a first-in-first-out *adaptor* (by default it wraps a `std::deque`): `emplace` / `push` at the back, `front()` to read and `pop()` to remove at the front. It's what breadth-first traversal needs. (A LIFO `std::stack` adaptor also exists. We use a `vector` as the stack instead because it's contiguous and lets us `reserve`; both are fine choices.)
- **`std::pair<A, B>`** (`<utility>`) holds two values, `.first` and `.second`. We use it for "(node, level)" and "(source node, its copy)".
- **Structured bindings** (C++17) unpack a pair, tuple or struct into named variables:

  ```
  auto [node, level] = queue.front();        // copies the two values out
  auto& [current, nextChild] = stack.back(); // binds references to the element's members
  ```

  Use the `auto&` form when you need to *modify* the element in place (as `postOrder` does with `++nextChild`).

C25. Algorithms: `std::find_if`, `std::max`, `std::accumulate`

The `<algorithm>` and `<numeric>` headers hold generic algorithms that work on any iterator range. **Prefer a named algorithm to a hand-written loop.** The name states the intent, and the implementation is tested.

- `std::find_if(first, last, pred)` returns an iterator to the first element where `pred` is true, or `last` if there is none. `detach` uses it to locate the `unique_ptr` that owns a given node among its siblings.
- `std::max(a, b)` returns the larger value.
- `std::accumulate(first, last, init)` (`<numeric>`) returns `init` plus every element. The tests use it to check that our iterators work with standard algorithms.

G. Functions as values

C26. Lambdas, captures, and how to accept a callable

A **lambda** is an unnamed function object written inline:

```
auto isBig = [](int v) { return v > 4; };              // no captures
std::size_t count = 0;
auto countOne = [&count](const Node&) { ++count; };    // captures `count` by reference
auto matches = [&target](const T& v) { return v == target; };
auto generic = [](const auto& n) { /* ... */ };        // "generic lambda": its parameter type is deduced
auto asRef = [](const std::unique_ptr<Node>& p) -> Node& { return *p; }; // explicit return type
```

- **Captures:** `[x]` copies `x`, `[&x]` refers to it, `[&]` / `[=]` capture everything used, by reference / by copy. Prefer to name what you capture. A reference capture must not outlive the variable it refers to.
- A lambda's type is unique and unnamed, so you store it in `auto` or pass it to a template.
- Why write `-> Node&` in `asRef`: without it, the lambda deduces the return type `Node` and returns a *copy* of the node. `Node` is non-copyable, so that wouldn't even compile.

How should a member function accept a callable? There are several good options:

Option	Signature	Trade-off
Template parameter	`template <typename Visit> void forEachPreOrder(Visit&& visit)`	zero overhead; the compiler can inline the call. Must live in a header. **Used.**
Constrained template	`template <std::invocable<Node&> Visit> ...`	as above, plus a clear error when the callable doesn't fit
`std::function<void(Node&)>` (`<functional>`)	ordinary parameter	can be stored and passed through non-template code; costs a type-erased indirect call and possibly a heap allocation
Function pointer	`void (*visit)(Node&)`	no captures possible

When a template helper takes the callable by lvalue reference (`Visit& visit`), callers must pass a *named* lambda. A temporary lambda is an rvalue and won't bind to a non-const `&`. That's why `countNodes` names its lambda `countOne` before passing it on.

H. Iterators and ranges

C27. Iterators and writing your own

An **iterator** is a generalised pointer: `*it` gives the element, `++it` moves on, and `it == end` says you've finished. Everything in `<algorithm>`, the range-based `for` loop and container constructors like `std::vector<int>(first, last)` work on iterators, so **a container that provides `begin()` / `end()` works with the whole standard library for free.**

Iterator **categories** say what an iterator supports: *input* (one pass only), *forward* (multi-pass: a copy iterates independently), *bidirectional* (`--`), *random access* (`it + n`), and *contiguous*. A tree traversal is naturally **forward**. (Going backwards in pre-order is possible with parent pointers, but not worth the complexity here.)

A C++20 forward iterator must provide:

```
using iterator_concept  = std::forward_iterator_tag; // C++20 algorithms read this
using iterator_category = std::forward_iterator_tag; // pre-C++20 code reads this
using value_type        = T;                         // the element type, without const or &
using difference_type   = std::ptrdiff_t;            // C4
using reference         = T&;                        // what * returns
using pointer           = T*;                        // what -> returns
Iterator();                                          // default-constructible
reference operator*() const;
pointer   operator->() const;
Iterator& operator++();                              // prefix
Iterator  operator++(int);                           // postfix
friend bool operator==(const Iterator&, const Iterator&);
```

plus being copyable. The tag types come from `<iterator>`. Verify the whole list at compile time with `static_assert(std::forward_iterator<It>)` (C21, C29). This is far better than finding a missing piece when some algorithm refuses to compile.

The range-based `for` loop `for (auto& v : tree) { ... }` is shorthand for calling `begin()` / `end()` and looping with `!=` and `++`.

Provide `const` iteration too: `begin() const` returns a `const_iterator`, so code holding a `const Tree&` can read but not modify. `cbegin()` / `cend()` force the const version even on a mutable tree.

C28. Ranges and views (C++20)

`<ranges>` adds **views**: lightweight, *lazy* adaptors over a range. `std::views::transform(f)` presents each element passed through `f`, without copying anything or allocating:

```
auto children()
{
    return mChildren | std::views::transform(
        [](const std::unique_ptr<Node>& p) -> Node& { return *p; });
}
// for (Node& child : node.children()) { ... }   -> the caller never sees a unique_ptr
```

The `|` "pipe" applies the adaptor to the range on its left. `auto` return type deduction (C++14) lets the function return the complicated view type without naming it.

Options for exposing a node's children:

Option	Pros	Cons
Return `const std::vector<std::unique_ptr<Node>>&`	trivial	leaks the storage choice into every caller; can't change it later
Index API: `childCount()` + `child(i)`	simple, stable	loop by index only
Transform view `children()`	range-for over `Node&`; hides `unique_ptr`	needs C++20
A custom child iterator class	full control	the most code

We provide the index API **and** the view. (`std::generator`, a C++23 coroutine-based way to write traversals as simple loops, needs GCC 14+.)

I. Errors and correctness

C29. `assert`, `NDEBUG`, and `static_assert`

- **`assert(cond)`** (`<cassert>`) aborts with a message if `cond` is false. It's **compiled out entirely** when the macro `NDEBUG` is defined, which release builds do (`-DNDEBUG`). Use it for **preconditions that are the caller's bug**: "you passed a node that isn't in this tree". It costs nothing in release.

  Cost warning, measured while writing this document: `contains()` is O(depth). Asserting it in every `addChild` makes building a very deep tree $O(n^2)$ in debug builds. Building a million-deep chain didn't finish within two minutes under the sanitizers, but takes 0.16 s in release. Debug-only checks may be slow, but know when they are.

- **Never put side effects in an `assert`.** `assert(x = compute())` vanishes in release, taking the side effect with it.

- **`static_assert(cond)`** is checked at *compile time* and costs nothing at run time. Use it to lock in type-level promises ("the iterator is a forward iterator", "Tree's move is noexcept").

C30. Exceptions and exception-safety guarantees

`throw std::logic_error("message");` exits the current function, destroys every local object on the way out (RAII, C13), and continues at the nearest matching `try { … } catch (const std::logic_error& e) { … }`. Standard exception types (`<stdexcept>`):

Type	Meaning	We throw it when
`std::logic_error`	the program misused the API in a detectable way	`emplaceRoot` on a tree that already has a root
`std::invalid_argument` (a `logic_error`)	an argument's value is unacceptable	`attach` with an empty tree, or with the tree itself
`std::out_of_range` (a `logic_error`)	an index is outside the valid range	`child(i)`, via `vector::at`

Catch by `const&`: catching by value copies the exception and can *slice* it (keep only the base-class part).

Exception-safety guarantees. Every operation should state which one it gives:

Guarantee	Promise	Examples in `Tree`
No-throw	never throws (`noexcept`)	move, swap, `size` , `empty` , `contains` , destructor
Strong	if it throws, the object is exactly as it was before (“commit or roll back”)	`emplaceChild` , `attach` , `detach` , copy assignment
Basic	if it throws, no leaks, and invariants still hold, but the contents may have changed	(the weakest acceptable level)

Technique for the strong guarantee: **do everything that can throw first, into temporary objects; then commit with operations that can’t throw.** `emplaceChild` builds the node, then reserves space (both can throw, and nothing observable has changed yet), then links the node in with operations that can’t throw.

C31. Choosing how to report errors

Mechanism	Use when	Tree example
`assert` (precondition)	a caller’s bug; checking it in release is too expensive	node belongs to this tree
Exception	a misuse or failure you can detect and the caller can handle, possibly far up the stack	root already exists
`nullptr` / `std::optional<T>` (`<optional>`)	“not found” or “absent” is a *normal* outcome, not an error	`find` returns `nullptr` ; `root()` on an empty tree
`std::expected<T, E>` (C++23)	failure is normal and the caller needs to know *why*	parsing a tree from text
Error codes / `bool`	C interop, or exceptions are disabled (some game/embedded code)	`Trie::addWord` in this repo returns `bool`

Options for “the node might not exist”: return `Node*` (nullable; **used**, because that’s the natural null for a handle); return `Node&` and make non-emptiness a precondition; return `std::optional<std::reference_wrapper<Node>>` (explicit, but verbose).

Part 2 — Designing the tree

2.1 Vocabulary

- **Root:** the one node with no parent. **Leaf:** a node with no children.
- **Parent / child / sibling:** direct links. **Ancestor / descendant:** links followed transitively.
- **Subtree of `n`:** `n` and all of its descendants. Any subtree is itself a tree.
- **Depth of `n`:** the number of edges from the root to `n` (the root has depth 0).
- **Height of a tree:** we define it as the **number of levels**: 0 for an empty tree, 1 for a lone root. Another common definition is the number of edges on the longest path (−1 for empty, 0 for a lone root). Pick one, write it down, and stay consistent.
- A tree with *n* nodes always has *n − 1* edges, and there is exactly one path between any two nodes.

2.2 The operations a tree needs

1. **Construction:** empty, with a root, from another tree (copy), from a temporary (move).
2. **Growth:** add a child under a given node; build values in place.
3. **Removal:** remove a subtree; remove everything.
4. **Restructuring:** move a subtree elsewhere (`detach` + `attach`).
5. **Navigation:** from a node, to its parent and children.
6. **Queries:** size, emptiness, height, depth, search.
7. **Traversal:** visit every node in a defined order.
8. **Value semantics:** copy, move, swap, compare.

2.3 Representation: how nodes are stored

Option	Sketch	Strengths	Weaknesses
A. Node owns a vector of child pointers	`struct Node { T v; Node* parent; std::vector<std::unique_ptr<Node>> kids; };`	direct child access by index; stable node addresses; subtrees move in O(1)	one heap allocation per node, plus one per children vector
B. First-child / next-sibling	`struct Node { T v; std::unique_ptr<Node> firstChild, nextSibling; Node* parent; };`	any N-ary tree using only two links; one allocation per node	reaching the k-th child is O(k)
C. Arena + indices	`std::vector<NodeData> nodes;` with `std::size_t parent; std::vector<std::size_t> kids;`	cache-friendly; one big allocation; trivially copyable when `T` is; indices survive copying	erasing leaves holes (needs a free list); indices must be validated
D. Parent array	`std::vector<int> parent;` (`parent[i]` is `i`'s parent)	tiny; ideal for union-find and competitive programming	finding children needs a scan or a second structure
E. Adjacency list	`std::vector<std::vector<int>> adj;`	the standard for trees given as edge lists (the CSES and Codeforces style you practise)	no root/parent unless you compute it; not a reusable container

Choice: A. It's the most general, it maps directly onto the concepts, and user handles stay valid. Part 6 sketches B and C. For contest problems, E is almost always the right tool; this document is about a reusable container.

2.4 Ownership: who deletes each node?

Option	Parent → child	Child → parent	Verdict
`unique_ptr` down, raw pointer up	`std::unique_ptr`	`Node*` (non-owning)	**Used.** Ownership matches the structure (each node has exactly one owner, its parent), and costs nothing.
`shared_ptr` down, `weak_ptr` up	`std::shared_ptr`	`std::weak_ptr`	Useful only when outside code must keep a node alive after it's removed from the tree. Pays for reference counts. A `shared_ptr` from child to parent would form a cycle and leak.
Raw `new` / `delete`	`Node*`	`Node*`	Don't. You'd have to write the RAII by hand and get it exactly right.
Arena (option C)	indices	indices	Ownership is simply "the tree owns the vector".

2.5 How users refer to nodes

Users need a way to say "add a child *here*". The options:

1. **`Node&` / `Node*` handles (used).** They are natural and cheap. They stay valid until *that* node is erased, because nodes never move in memory (C23). The precondition "the handle belongs to this tree" is checked by `assert(contains(node))` in debug builds.
2. **Opaque ids** (`NodeId` wrapping an index). Survive copying the tree; natural for the arena design.
3. **Iterators as positions**, like `std::list::insert(it, v)`. More work to design.

`Node`'s constructor and fields are private (C11), so a user can hold and read a `Node` but can't forge one or break the parent links.

2.6 Where functionality lives: member vs non-member

Prefer non-member, non-friend functions when a function can be written using only the public interface. They don't depend on private details, so changing the internals can't break them, and the class stays small. That's why `printTree` in the tests is a free function. A member function is justified when it needs private data or preserves an invariant, as `detach` and `attach` do.

2.7 Traversal orders, and recursion vs iteration

- **Pre-order:** node, then its children left to right. Good for copying a tree, or printing it with indentation.
- **Post-order:** children first, then the node. Good for deleting, and for computing a value from the children's values (subtree sizes, expression evaluation).
- **Level-order (BFS):** level by level, using a queue (C24). Good for height, and for finding the nodes nearest the root first.
- **In-order:** left subtree, node, right subtree. Only defined for binary trees (Part 6.1); on a BST it yields sorted order.

Recursive traversal is the textbook version, and short:

```
void preOrder(Node* n, auto& visit) {
    if (!n) return;
    visit(*n);
    for (auto& c : n->mChildren) preOrder(c.get(), visit);
}
```

But it uses one call-stack frame per level of depth (C13). **Iterative** traversal replaces the call stack with an explicit `std::vector` stack on the heap. It is limited only by memory and is **used throughout**. (A middle option: recurse, and document a maximum depth.)

Options for how users traverse: callbacks (`forEachPreOrder(visit)`), which are simple, work in every order and can pass extra information such as the level; iterators, which plug into range-for and `<algorithm>`; ranges or generators. We give callbacks for all three orders and an iterator for pre-order.

2.8 Destruction, and the recursive-destructor trap

With `unique_ptr` children, the compiler-generated destructor is correct but **recursive**. Destroying a node destroys its children vector, which destroys each `unique_ptr`, which destroys each child node, and so on. A tree shaped like a long chain (a degenerate BST, or a linked-list-like tree) overflows the stack. Tested while writing this document: a minimal `unique_ptr` chain one million nodes deep **segfaults** with the default 8 MB stack.

Fix: `clear()` detaches nodes into a heap-allocated work list and destroys each one *after* moving its children out, so no destructor ever recurses. `~Tree()` calls `clear()`. The copy constructor and `operator==` are iterative for the same reason.

2.9 Exception guarantees chosen for each operation

See C30 for the definitions and Part 4 for the code. One subtlety: `clear()` is `noexcept` but allocates its work list, so running out of memory *during destruction* ends the program. Destructors must not throw anyway, and running out of memory in the middle of destruction can't be recovered from sensibly, so that's an accepted trade-off. (It's possible to destroy without allocating, by rotating the tree into a chain first. It's more complex, and an exercise in Part 7.)

2.10 Complexity

For *n* nodes, height *h*, a node with *k* children and depth *d*:

Operation	Time	Notes
`size`, `empty`, `root`	O(1)	size is cached in `mSize`
`addChild`, `emplaceChild`	amortised O(1)	plus O(d) for the debug-only `contains` assert
`detach`, `erase`	O(k + subtree size)	finding the sibling, plus counting the subtree to keep `mSize` right
`attach`	amortised O(1)	
`contains`, `depth`	O(d)	walks parent pointers
`height`, traversals, `find`, copy, `==`, `clear`	O(n)	extra memory up to O(n) for the work stack or queue
move, swap	O(1)	

Design option: store each subtree's size in its node, which makes `detach` O(k). The cost is that every insert must update all ancestors, which is O(d).

2.11 Thread safety

This follows the same rule as the standard containers: any number of threads may call `const` functions at the same time; any modification needs exclusive access (for example, a `std::mutex` held by the caller). A container shouldn't lock internally; the caller knows the right granularity.

Part 3 — Class-design checklist

Good C++ class-design practices, each mapped to where `Tree` applies it.

1. **State the invariants** in a comment at the top of the class, and keep the data `private` so only member functions can change it (C5).
2. **Initialize every member**, preferably with default member initializers (C6).
3. **Make single-argument constructors** `explicit` (C6).
4. **Decide the special members deliberately:** rule of zero if you can, rule of five if you must, and say which one in the code (C15).
5. **Give value semantics** (deep copy, cheap move) to anything that acts like a value or a container (C15, C16).
6. **Mark moves, swap and destructors** `noexcept` (C8).
7. **Be const-correct:** `const` on member functions that don't modify, and `const`/non-`const` overload pairs for accessors (C8).
8. **Use** `[[nodiscard]]` where ignoring the result is almost certainly a bug (C8).
9. **Express ownership in types:** `unique_ptr` owns, raw pointers and references observe, and there is no naked `new`/`delete` outside one contained helper (C14).
10. **Hide representation:** users see `Node&`, values and views, never the `unique_ptr`s (C28).
11. **Follow standard-library conventions:** `size()`, `empty()`, `begin()`/`end()`, `value_type`, `iterator`. Generic code and other programmers already understand them (C9, C27).
12. **Constrain templates with concepts** (C21) and **lock in promises with** `static_assert` (C29).
13. **Document each operation's preconditions, complexity and exception guarantee** (Part 2.9–2.10).
14. **Prefer non-member non-friend functions** when the public interface is enough (Part 2.6); write `swap` and operators as hidden friends (C11).
15. **Reuse the standard library** (algorithms, containers, utilities) instead of re-implementing it (C23–C25).
16. **Watch the stack:** there's no unbounded recursion over user-controlled depth (Part 2.8).
17. **Test the edges:** empty trees, single nodes, copies being independent, moved-from state, deep trees, and the error paths (Part 5).
18. **Build with warnings as errors and with sanitizers** (Part 0).

Part 4 — The implementation

4.1 Headers used, and what for

Header	Names used
`<algorithm>`	`std::find_if`, `std::max` (C25)
`<cassert>`	`assert` (C29)
`<concepts>`	`std::predicate`, `std::equality_comparable` (C21)
`<cstddef>`	`std::size_t`, `std::ptrdiff_t` (C4)
`<iterator>`	`std::forward_iterator_tag` (C27)
`<memory>`	`std::unique_ptr` (C14)
`<queue>`	`std::queue` (C24)
`<ranges>`	`std::views::transform` (C28)
`<stdexcept>`	`std::logic_error`, `std::invalid_argument` (C30)
`<type_traits>`	`std::conditional_t` (C20)
`<utility>`	`std::move`, `std::forward`, `std::exchange`, `std::swap`, `std::pair` (C15, C16, C22, C24)
`<vector>`	`std::vector` (C23)

4.2 Walkthrough

Read the code below with these notes:

- **Class comment and invariants** (C5). Every public function either keeps the three invariants or changes them only in `mSize`/links it fixes up before returning.
- **`Node`** (C9, C7, C11, C8, C28). It's non-copyable. It has a private constructor that forwards arguments to build `T` in place (C22), and `friend class Tree`. Its accessors come in const/non-const pairs. `child(i)` delegates bounds checking to `vector::at`. `children()` returns a transform view.
- **`PreOrderIterator<IsConst>`** (C20, C27, C12). A vector acts as the explicit stack; the top of the stack is the current node. `operator++` pops it and pushes its children in reverse (C23 `rbegin`), so the leftmost child comes out next. Two iterators are equal when they point at the same current node, and every exhausted iterator (empty stack) equals the default-built `end()`. Copying an iterator copies its stack, so copies advance independently. That's the multi-pass property forward iterators require.
- **Constructors** (C6, C7, C18). `Tree(T rootValue)` is `explicit` and takes its argument by value as a sink.
- **Copy constructor** (C15, C24, C23). Iterative. It walks pairs of (original node, its copy) and reserves each children vector before filling it. If an allocation throws halfway, the partly built `mRoot` is a fully constructed member, so it's destroyed automatically and nothing leaks.
- **Move constructor** (C16). O(1), `noexcept`, and uses `std::exchange`.
- **Copy-and-swap `operator=`** (C17). It's the only assignment operator.
- **`swap`**, a hidden friend (C11, C15).
- **`emplaceRoot`** (C22, C30). It throws `std::logic_error` if a root exists. *Option:* make it replace the existing tree instead, or return `bool` like `Trie::addWord`.
- **`emplaceChild`** (C22, C23, C30). Build, reserve, then commit: the strong guarantee. `addChild` has the two overloads from C18.
- **`detach`** (C23, C25, C16, C8). It counts the subtree *first* (that can throw), then commits by moving the `unique_ptr` out and erasing the empty slot. It's `[[nodiscard]]` because ignoring the result destroys the subtree. Detaching the root moves the whole tree out: `return std::move(*this);`.
- **`erase`** reuses `detach` and discards the result with `static_cast<void>` (C10).
- **`attach`** (C30). It validates its arguments and reserves before committing. `std::exchange` leaves the source tree empty.
- **`clear`** (Part 2.8). Iterative destruction.
- **`contains` / `depth`** (C8). `contains` is `noexcept`; `depth` is also `static`.
- **`height`** is built on `forEachLevel`: reuse, not a new loop.
- **`findIf` / `find`** (C21, C26). They're constrained by concepts and stop early. `find` exists only when `T` has `==`.
- **Traversals** (C8, C26). Public const/non-const pairs forward to private static templates parameterised on `NodePtr`, so there's one body per traversal.
- **`operator==`** (C11, C12, C21). A hidden friend, constrained with a `requires`-clause, and iterative. `!=` comes free in C++20.

- `makeNode` is the only place a `new` appears (C14).
- `postOrder` keeps one frame per node on its explicit stack: the node, plus how many of its children have finished. Note the reference-invalidation care described in C23.

4.3 `Tree.hpp`

```cpp
#pragma once

#include <algorithm>
#include <cassert>
#include <concepts>
#include <cstddef>
#include <iterator>
#include <memory>
#include <queue>
#include <ranges>
#include <stdexcept>
#include <type_traits>
#include <utility>
#include <vector>

namespace dsa
{

////////////////////////////////////////////////////////////////////////////
// A generic rooted tree in which every node owns any number of children.
//
// Invariants (true after every public member function returns):
//   1. mRoot == nullptr  <=>  mSize == 0.
//   2. Every node's mParent points at the node whose mChildren owns it;
//      the root's mParent is nullptr.
//   3. mSize equals the number of nodes reachable from mRoot.
template <typename T>
class Tree
{
public:
    ////////////////////////////////////////////////////////////////////////
    // One node of the tree. Users receive Node& handles from the tree but
    // can never create, copy or destroy a Node themselves.
    class Node
    {
    public:
        Node(const Node&) = delete;
        Node& operator=(const Node&) = delete;

        // Getters.
        [[nodiscard]] T& value() noexcept { return mValue; }
        [[nodiscard]] const T& value() const noexcept { return mValue; }
        [[nodiscard]] Node* parent() noexcept { return mParent; }
        [[nodiscard]] const Node* parent() const noexcept { return mParent; }
        [[nodiscard]] std::size_t childCount() const noexcept { return mChildren.size(); }
        [[nodiscard]] bool isLeaf() const noexcept { return mChildren.empty(); }
        [[nodiscard]] bool isRoot() const noexcept { return mParent == nullptr; }

        // Bounds-checked access to the i-th child; throws std::out_of_range.
        [[nodiscard]] Node& child(std::size_t i) { return *mChildren.at(i); }
        [[nodiscard]] const Node& child(std::size_t i) const { return *mChildren.at(i); }

        // A lazy view over the children as Node& (hides the unique_ptrs).
        [[nodiscard]] auto children()
        {
            return mChildren | std::views::transform(
                [](const std::unique_ptr<Node>& p) -> Node& { return *p; });
        }
        [[nodiscard]] auto children() const
        {
            return mChildren | std::views::transform(
                [](const std::unique_ptr<Node>& p) -> const Node& { return *p; });
        }

    private:
        friend class Tree;

        template <typename... Args>
        explicit Node(Node* parent, Args&&... args)
            : mValue(std::forward<Args>(args)...), mParent(parent)
        {
        }

        T mValue;
        Node* mParent{nullptr};                      // non-owning
        std::vector<std::unique_ptr<Node>> mChildren; // owning
```

```
};

////////////////////////////////////////////////////////////////////////
// Pre-order (node, then children left to right) forward iterator over
// the stored values. IsConst selects iterator vs const_iterator.
template <bool IsConst>
class PreOrderIterator
{
    using NodePtr = std::conditional_t<IsConst, const Node*, Node*>;

public:
    using iterator_concept = std::forward_iterator_tag;
    using iterator_category = std::forward_iterator_tag;
    using value_type = T;
    using difference_type = std::ptrdiff_t;
    using reference = std::conditional_t<IsConst, const T&, T&>;
    using pointer = std::conditional_t<IsConst, const T*, T*>;

    PreOrderIterator() = default;
    explicit PreOrderIterator(NodePtr root)
    {
        if (root != nullptr)
        {
            mStack.push_back(root);
        }
    }

    reference operator*() const { return mStack.back()->mValue; }
    pointer operator->() const { return &mStack.back()->mValue; }

    PreOrderIterator& operator++()
    {
        NodePtr current = mStack.back();
        mStack.pop_back();
        // Push in reverse so the leftmost child is visited first.
        for (auto it = current->mChildren.rbegin(); it != current->mChildren.rend(); ++it)
        {
            mStack.push_back(it->get());
        }
        return *this;
    }

    PreOrderIterator operator++(int)
    {
        PreOrderIterator copy = *this;
        ++*this;
        return copy;
    }

    friend bool operator==(const PreOrderIterator& a, const PreOrderIterator& b) noexcept
    {
        return a.current() == b.current();
    }

private:
    NodePtr current() const noexcept { return mStack.empty() ? nullptr : mStack.back(); }

    std::vector<NodePtr> mStack;
};

using value_type = T;
using size_type = std::size_t;
using iterator = PreOrderIterator<false>;
using const_iterator = PreOrderIterator<true>;

// Constructors, assignment, destructor (rule of five).
Tree() = default;
explicit Tree(T rootValue) { emplaceRoot(std::move(rootValue)); }

Tree(const Tree& other)
{
    if (other.empty())
    {
        return;
    }
    mRoot = makeNode(nullptr, other.mRoot->mValue);
    // Pairs of (node to copy from, its already-made copy).
    std::vector<std::pair<const Node*, Node*>> pending{{other.mRoot.get(), mRoot.get()}};
    while (!pending.empty())
    {
```

```
            auto [source, copy] = pending.back();
            pending.pop_back();
            copy->mChildren.reserve(source->mChildren.size());
            for (const std::unique_ptr<Node>& child : source->mChildren)
            {
                copy->mChildren.push_back(makeNode(copy, child->mValue));
                pending.emplace_back(child.get(), copy->mChildren.back().get());
            }
        }
        mSize = other.mSize;
    }

    Tree(Tree&& other) noexcept
        : mRoot(std::move(other.mRoot)), mSize(std::exchange(other.mSize, 0))
    {
    }

    // Copy-and-swap: one operator serves both copy- and move-assignment.
    Tree& operator=(Tree other) noexcept
    {
        swap(*this, other);
        return *this;
    }

    ~Tree() { clear(); }

    friend void swap(Tree& a, Tree& b) noexcept
    {
        using std::swap;
        swap(a.mRoot, b.mRoot);
        swap(a.mSize, b.mSize);
    }

    // Capacity.
    [[nodiscard]] bool empty() const noexcept { return mRoot == nullptr; }
    [[nodiscard]] size_type size() const noexcept { return mSize; }

    // Access. nullptr when the tree is empty.
    [[nodiscard]] Node* root() noexcept { return mRoot.get(); }
    [[nodiscard]] const Node* root() const noexcept { return mRoot.get(); }

    // Modifiers.
    // Create the root in place; throws std::logic_error if one exists.
    template <typename... Args>
    Node& emplaceRoot(Args&&... args)
    {
        if (!empty())
        {
            throw std::logic_error("Tree::emplaceRoot: tree already has a root");
        }
        mRoot = makeNode(nullptr, std::forward<Args>(args)...);
        mSize = 1;
        return *mRoot;
    }

    // Append a new last child of `parent`, built in place from args.
    // Precondition: parent belongs to this tree. Strong exception guarantee.
    template <typename... Args>
    Node& emplaceChild(Node& parent, Args&&... args)
    {
        assert(contains(parent));
        std::unique_ptr<Node> node = makeNode(&parent, std::forward<Args>(args)...);
        parent.mChildren.reserve(parent.mChildren.size() + 1); // may throw; nothing changed yet
        parent.mChildren.push_back(std::move(node));           // cannot throw now
        ++mSize;
        return *parent.mChildren.back();
    }

    Node& addChild(Node& parent, const T& value) { return emplaceChild(parent, value); }
    Node& addChild(Node& parent, T&& value) { return emplaceChild(parent, std::move(value)); }

    // Remove the subtree rooted at `node` and return it as its own tree.
    // Node& handles into the subtree stay valid and now belong to the result.
    [[nodiscard]] Tree detach(Node& node)
    {
        assert(contains(node));
        if (node.isRoot())
        {
            return std::move(*this);
        }
```

```
    const size_type detachedSize = countNodes(&node); // may throw; nothing changed yet
    auto& siblings = node.mParent->mChildren;
    auto it = std::find_if(siblings.begin(), siblings.end(),
                           [&node](const std::unique_ptr<Node>& p) { return p.get() == &node; });
    Tree result;
    result.mRoot = std::move(*it);
    siblings.erase(it);
    result.mRoot->mParent = nullptr;
    result.mSize = detachedSize;
    mSize -= detachedSize;
    return result;
}

// Remove and destroy the subtree rooted at `node`.
void erase(Node& node) { static_cast<void>(detach(node)); }

// Move a whole tree in as the last child of `parent`. Strong guarantee.
Node& attach(Node& parent, Tree&& subtree)
{
    assert(contains(parent));
    if (subtree.empty())
    {
        throw std::invalid_argument("Tree::attach: subtree is empty");
    }
    if (&subtree == this)
    {
        throw std::invalid_argument("Tree::attach: cannot attach a tree to itself");
    }
    parent.mChildren.reserve(parent.mChildren.size() + 1);
    subtree.mRoot->mParent = &parent;
    parent.mChildren.push_back(std::move(subtree.mRoot));
    mSize += std::exchange(subtree.mSize, 0);
    return *parent.mChildren.back();
}

// Destroy every node without recursion, so a very deep tree cannot
// overflow the call stack.
void clear() noexcept
{
    std::vector<std::unique_ptr<Node>> pending;
    if (mRoot != nullptr)
    {
        pending.push_back(std::move(mRoot));
    }
    while (!pending.empty())
    {
        std::unique_ptr<Node> node = std::move(pending.back());
        pending.pop_back();
        for (std::unique_ptr<Node>& child : node->mChildren)
        {
            pending.push_back(std::move(child));
        }
        // `node` is destroyed here; its children were moved out, so
        // its destructor does not recurse.
    }
    mSize = 0;
}

// Queries.
// True if `node` belongs to this tree. O(depth of node).
[[nodiscard]] bool contains(const Node& node) const noexcept
{
    const Node* top = &node;
    while (top->mParent != nullptr)
    {
        top = top->mParent;
    }
    return top == mRoot.get();
}

// Number of edges between `node` and the root of its tree.
[[nodiscard]] static size_type depth(const Node& node) noexcept
{
    size_type edges = 0;
    for (const Node* p = node.mParent; p != nullptr; p = p->mParent)
    {
        ++edges;
    }
    return edges;
}
```

```
    // Number of levels (0 for an empty tree, 1 for a lone root).
    [[nodiscard]] size_type height() const
    {
        size_type levels = 0;
        forEachLevel([&levels](const Node&, size_type level) { levels = std::max(levels, level + 1); });
        return levels;
    }

    // First node, in pre-order, whose value satisfies `pred`; else nullptr.
    template <std::predicate<const T&> Pred>
    [[nodiscard]] Node* findIf(Pred pred) { return findIfFrom(mRoot.get(), pred); }
    template <std::predicate<const T&> Pred>
    [[nodiscard]] const Node* findIf(Pred pred) const
    {
        return findIfFrom(static_cast<const Node*>(mRoot.get()), pred);
    }

    // First node, in pre-order, whose value equals `target`; else nullptr.
    [[nodiscard]] Node* find(const T& target)
        requires std::equality_comparable<T>
    {
        return findIf([&target](const T& v) { return v == target; });
    }
    [[nodiscard]] const Node* find(const T& target) const
        requires std::equality_comparable<T>
    {
        return findIf([&target](const T& v) { return v == target; });
    }

    // Traversals. `visit` is called with each Node (const Node& from a const tree).
    template <typename Visit>
    void forEachPreOrder(Visit&& visit) { preOrder(mRoot.get(), visit); }
    template <typename Visit>
    void forEachPreOrder(Visit&& visit) const { preOrder(static_cast<const Node*>(mRoot.get()), visit); }

    template <typename Visit>
    void forEachPostOrder(Visit&& visit) { postOrder(mRoot.get(), visit); }
    template <typename Visit>
    void forEachPostOrder(Visit&& visit) const { postOrder(static_cast<const Node*>(mRoot.get()), visit); }

    // Breadth-first; `visit` also receives the level (root = 0).
    template <typename Visit>
    void forEachLevel(Visit&& visit) { levelOrder(mRoot.get(), visit); }
    template <typename Visit>
    void forEachLevel(Visit&& visit) const { levelOrder(static_cast<const Node*>(mRoot.get()), visit); }

    // Iteration (pre-order over values).
    [[nodiscard]] iterator begin() noexcept { return iterator(mRoot.get()); }
    [[nodiscard]] iterator end() noexcept { return iterator(); }
    [[nodiscard]] const_iterator begin() const noexcept { return const_iterator(mRoot.get()); }
    [[nodiscard]] const_iterator end() const noexcept { return const_iterator(); }
    [[nodiscard]] const_iterator cbegin() const noexcept { return begin(); }
    [[nodiscard]] const_iterator cend() const noexcept { return end(); }

    // Structural equality: same shape and equal values in the same places.
    friend bool operator==(const Tree& a, const Tree& b)
        requires std::equality_comparable<T>
    {
        if (a.mSize != b.mSize)
        {
            return false;
        }
        if (a.empty())
        {
            return true;
        }
        std::vector<std::pair<const Node*, const Node*>> pending{{a.mRoot.get(), b.mRoot.get()}};
        while (!pending.empty())
        {
            auto [x, y] = pending.back();
            pending.pop_back();
            if (!(x->mValue == y->mValue) || x->mChildren.size() != y->mChildren.size())
            {
                return false;
            }
            for (size_type i = 0; i < x->mChildren.size(); ++i)
            {
                pending.emplace_back(x->mChildren[i].get(), y->mChildren[i].get());
            }
```

```
        }
        return true;
    }

private:
    template <typename... Args>
    static std::unique_ptr<Node> makeNode(Node* parent, Args&&... args)
    {
        // std::make_unique cannot reach Node's private constructor, so we
        // call `new` ourselves and hand the pointer straight to unique_ptr.
        return std::unique_ptr<Node>(new Node(parent, std::forward<Args>(args)...));
    }

    static size_type countNodes(const Node* start)
    {
        size_type count = 0;
        auto countOne = [&count](const Node&) { ++count; };
        preOrder(start, countOne);
        return count;
    }

    // NodePtr is Node* or const Node*, so one body serves both constnesses.
    template <typename NodePtr, typename Visit>
    static void preOrder(NodePtr start, Visit& visit)
    {
        if (start == nullptr)
        {
            return;
        }
        std::vector<NodePtr> stack{start};
        while (!stack.empty())
        {
            NodePtr current = stack.back();
            stack.pop_back();
            visit(*current);
            for (auto it = current->mChildren.rbegin(); it != current->mChildren.rend(); ++it)
            {
                stack.push_back(it->get());
            }
        }
    }

    template <typename NodePtr, typename Pred>
    static NodePtr findIfFrom(NodePtr start, Pred& pred)
    {
        if (start == nullptr)
        {
            return nullptr;
        }
        std::vector<NodePtr> stack{start};
        while (!stack.empty())
        {
            NodePtr current = stack.back();
            stack.pop_back();
            if (pred(current->mValue))
            {
                return current; // stop early: the rest is never visited
            }
            for (auto it = current->mChildren.rbegin(); it != current->mChildren.rend(); ++it)
            {
                stack.push_back(it->get());
            }
        }
        return nullptr;
    }

    template <typename NodePtr, typename Visit>
    static void postOrder(NodePtr start, Visit& visit)
    {
        if (start == nullptr)
        {
            return;
        }
        // Each frame remembers how many of its children are already done.
        std::vector<std::pair<NodePtr, size_type>> stack{{start, 0}};
        while (!stack.empty())
        {
            auto& [current, nextChild] = stack.back();
            if (nextChild < current->mChildren.size())
            {
```

```
                NodePtr child = current->mChildren[nextChild].get();
                ++nextChild;
                stack.emplace_back(child, 0); // may invalidate current/nextChild; not used after
            }
            else
            {
                NodePtr done = current;
                stack.pop_back();
                visit(*done);
            }
        }
    }

    template <typename NodePtr, typename Visit>
    static void levelOrder(NodePtr start, Visit& visit)
    {
        if (start == nullptr)
        {
            return;
        }
        std::queue<std::pair<NodePtr, size_type>> queue;
        queue.emplace(start, 0);
        while (!queue.empty())
        {
            auto [current, level] = queue.front();
            queue.pop();
            visit(*current, level);
            for (const auto& child : current->mChildren)
            {
                queue.emplace(child.get(), level + 1);
            }
        }
    }

    std::unique_ptr<Node> mRoot;
    size_type mSize{0};
};

} // namespace dsa
```

Part 5 — Tests

5.1 Tools used only by the tests

- **Macros** (`#define`) are text substitution done by the preprocessor before compilation. We define `CHECK(cond)` because `assert` disappears under `-DNDEBUG`, and the release build must still verify results. The body is wrapped in `do { … } while (false)` so that `CHECK(x);` behaves as a single statement, even inside an `if` with no braces. `#cond` turns the expression into a string; `__FILE__` and `__LINE__` give the source location. Outside tests like this, prefer functions or templates to macros: macros ignore scope and types.
- `std::cout` / `std::cerr` (`<iostream>`) are standard output and standard error. `std::ostream` (`<ostream>`) is the stream type `printTree` writes to.
- `std::exit(code)` and `std::atoi(text)` (`<cstdlib>`): end the program with an exit status, and parse an integer from a C string.
- `int main(int argc, char** argv)` : `argc` is the number of command-line arguments and `argv` the arguments themselves; `argv[1]` is the first one the user typed. The deep-chain test reads its depth from there.
- `std::string(3, 'a')` is `"aaa"` (the count-and-character constructor).
- `typename dsa::Tree<T>::Node` in `printTree` : inside a template, a name that depends on `T` and names a type must be prefixed with `typename`, so the compiler knows it's a type.

Test framework options: hand-rolled `CHECK` (no dependencies, **used**); GoogleTest (the industry standard, with fixtures and death tests); Catch2 and doctest (header-only, expressive). Use one of the frameworks once the tests outgrow one file.

5.2 What the tests cover

Structure and size; all three traversal orders; iterators with a standard algorithm (`std::accumulate`); the children view; `find` and `findIf` (found and not found); deep-copy independence and `==` / `!=` ; `detach` keeping handles valid and moving them to the new tree; `attach` emptying its source; `erase` ; the moved-from state; const-correctness, checked at compile time; building values in place; the `logic_error` path; compile-time checks of iterator concepts and `noexcept` move; and a deep chain that is copied, compared and destroyed without overflowing the stack.

5.3 `test.cpp`

```cpp
#include "Tree.hpp"

#include <cstdlib>
#include <iostream>
#include <iterator>
#include <numeric>
#include <ostream>
#include <string>
#include <vector>

// A non-member helper: it needs only the public interface, so it does not
// belong inside the class.
template <typename T>
void printTree(std::ostream& out, const dsa::Tree<T>& tree)
{
    tree.forEachPreOrder([&out](const typename dsa::Tree<T>::Node& node) {
        out << std::string(2 * dsa::Tree<T>::depth(node), ' ') << node.value() << '\n';
    });
}

// Unlike assert, CHECK stays active in -DNDEBUG builds.
#define CHECK(cond)                                                          \
    do                                                                       \
    {                                                                        \
        if (!(cond))                                                         \
        {                                                                    \
            std::cerr << __FILE__ << ':' << __LINE__ << ": CHECK failed: " #cond "\n"; \
            std::exit(1);                                                    \
        }                                                                    \
    } while (false)

static_assert(std::forward_iterator<dsa::Tree<int>::iterator>);
static_assert(std::forward_iterator<dsa::Tree<int>::const_iterator>);
static_assert(std::ranges::forward_range<dsa::Tree<int>>);
static_assert(!std::is_copy_constructible_v<dsa::Tree<int>::Node>);
static_assert(std::is_nothrow_move_constructible_v<dsa::Tree<int>>);

int main(int argc, char** argv)
{
    // 1
    // |-- 2
    // |   |-- 5
    // |   `-- 6
    // |-- 3
    // `-- 4
    //     `-- 7
    dsa::Tree<int> tree(1);
    auto& root = *tree.root();
    auto& two = tree.addChild(root, 2);
    tree.addChild(root, 3);
    auto& four = tree.addChild(root, 4);
    tree.addChild(two, 5);
    tree.addChild(two, 6);
    tree.addChild(four, 7);

    CHECK(tree.size() == 7);
    CHECK(tree.height() == 3);
    CHECK(dsa::Tree<int>::depth(two) == 1);

    std::vector<int> pre(tree.begin(), tree.end());
    CHECK((pre == std::vector<int>{1, 2, 5, 6, 3, 4, 7}));

    std::vector<int> post;
    tree.forEachPostOrder([&post](const auto& n) { post.push_back(n.value()); });
    CHECK((post == std::vector<int>{5, 6, 2, 3, 7, 4, 1}));

    std::vector<int> bfs;
    tree.forEachLevel([&bfs](const auto& n, std::size_t) { bfs.push_back(n.value()); });
    CHECK((bfs == std::vector<int>{1, 2, 3, 4, 5, 6, 7}));

    CHECK(std::accumulate(tree.begin(), tree.end(), 0) == 28);

    std::vector<int> kids;
    for (const auto& child : root.children())
    {
        kids.push_back(child.value());
```

```
    }
    CHECK((kids == std::vector<int>{2, 3, 4}));

    CHECK(tree.find(6) != nullptr && tree.find(6)->parent() == &two);
    CHECK(tree.find(42) == nullptr);
    CHECK(tree.findIf([](int v) { return v > 4; })->value() == 5);

    // Deep copy is independent of the original.
    dsa::Tree<int> copy = tree;
    CHECK(copy == tree);
    copy.root()->value() = 100;
    CHECK(tree.root()->value() == 1);
    CHECK(copy != tree);

    // Detach keeps handles valid; attach moves the subtree back in.
    dsa::Tree<int> branch = tree.detach(two);
    CHECK(tree.size() == 4 && branch.size() == 3);
    CHECK(branch.root() == &two && two.isRoot());
    CHECK(!tree.contains(two) && branch.contains(two));
    tree.attach(four, std::move(branch));
    CHECK(branch.empty() && tree.size() == 7);
    CHECK(dsa::Tree<int>::depth(two) == 2);

    tree.erase(four);
    CHECK(tree.size() == 2);

    // Move leaves the source empty.
    dsa::Tree<int> moved = std::move(tree);
    CHECK(moved.size() == 2 && tree.empty() && tree.begin() == tree.end());

    // A const tree hands out only const access.
    const dsa::Tree<int>& view = moved;
    static_assert(std::is_same_v<decltype(*view.begin()), const int&>);
    static_assert(std::is_same_v<decltype(view.root()), const dsa::Tree<int>::Node*>);

    // In-place construction of non-trivial values.
    dsa::Tree<std::string> words;
    auto& w = words.emplaceRoot(3, 'a'); // std::string(3, 'a')
    words.emplaceChild(w, "child");
    printTree(std::cout, words);
    try
    {
        words.emplaceRoot("again");
        CHECK(false);
    }
    catch (const std::logic_error&)
    {
    }

    // A very deep chain must not overflow the stack when destroyed or copied.
    // Pass the depth as argv[1]; keep it small in debug builds, where the
    // contains() assertion inside addChild makes each insertion O(depth).
    {
        const int chainDepth = argc > 1 ? std::atoi(argv[1]) : 10'000;
        dsa::Tree<int> chain(0);
        auto* tip = chain.root();
        for (int i = 1; i < chainDepth; ++i)
        {
            tip = &chain.addChild(*tip, i);
        }
        dsa::Tree<int> chainCopy = chain;
        CHECK(chainCopy.size() == static_cast<std::size_t>(chainDepth) &&
              chainCopy.height() == static_cast<std::size_t>(chainDepth));
        CHECK(chainCopy == chain);
    }

    std::cout << "all tests passed\n";
}
```

Output of both builds (from Part 0):

```
aaa
  child
all tests passed
```

Part 6 — Variants and when to use the standard library instead

6.1 Binary tree

Each node has at most two children, and "left" and "right" are distinct positions, even when only one exists:

```
struct BinaryNode
{
    T value;
    BinaryNode* parent{nullptr};
    std::unique_ptr<BinaryNode> left;   // two named slots instead of a vector
    std::unique_ptr<BinaryNode> right;
};
```

The class-design ideas carry over unchanged. In-order traversal becomes meaningful: left subtree, node, right subtree.

6.2 Binary search tree (BST)

A BST's invariant: every value in a node's left subtree is less than the node's value, and every value in its right subtree is greater. What changes:

- **Order comes from a comparator.** Take one as a template parameter, defaulting to `std::less<T>`, the function object for `<` in `<functional>`: `template <typename T, typename Compare = std::less<T>>`. Store it as a member; C++20's `[[no_unique_address]]` lets an empty comparator take no space.
- **Users must not get mutable access to values.** Changing a key in place would break the ordering invariant silently. That's why `std::set`'s iterators are always const. So drop `T& value()`.
- **Insertion position is decided by the tree,** not the caller: `insert(value)` instead of `addChild(parent, value)`.
- **Balance matters.** Inserting sorted data into a plain BST produces a chain: O(n) operations, and the deep-destruction problem from Part 2.8. Self-balancing trees (red-black, AVL, treaps) keep the height O(log n).

Use the standard library when you need ordered storage. `std::set`, `std::map`, `std::multiset` and `std::multimap` (`<set>`, `<map>`) are balanced BSTs (red-black trees in practice) with O(log n) insert, erase and lookup. They provide iterators, `lower_bound` / `upper_bound`, and node `extract` / `merge` (C++17) for moving nodes between containers without reallocating. Write your own BST to learn how they work, or when you need something they can't do (for example, order statistics: "the k-th smallest").

6.3 Arena (index-based) tree

```cpp
template <typename T>
class ArenaTree
{
public:
    using NodeId = std::size_t;                    // option: a strong type, struct NodeId { std::size_t v; };
    static constexpr NodeId npos = static_cast<NodeId>(-1);

    NodeId addChild(NodeId parent, T value)
    {
        const NodeId id = mNodes.size();
        mNodes.push_back({std::move(value), parent, {}});
        if (parent != npos) { mNodes[parent].children.push_back(id); }
        return id;
    }

private:
    struct Data { T value; NodeId parent; std::vector<NodeId> children; };
    std::vector<Data> mNodes;                      // one owner; copy/move/destroy by rule of zero
};
```

New pieces here: `constexpr` means "a constant known at compile time", and `static constexpr` makes it one shared constant for the class. The design **gets the rule of zero for free**: the vector does all the copying, moving and destruction without recursion, and access has good cache locality. The price: `Data&` references dangle when `mNodes` grows (C23), so users must hold ids; and erasing needs either a free list of reusable slots or compaction plus id remapping.

6.4 Trees implicit in arrays

Some trees need no nodes at all:

- **Binary heap:** node `i` has children `2i+1` and `2i+2`. The standard library provides it: `std::priority_queue` (`<queue>`), and `std::make_heap` / `push_heap` / `pop_heap` (`<algorithm>`).
- **Segment tree and Fenwick tree:** array-based trees for range queries (common in contests).
- **Trie:** a tree keyed by characters; see `everything-cpp/data-structures-and-algorithms/Trie.hpp`. A good next step is to apply this checklist to that class: ownership of `mChildren`, const-correctness, and the rule of five.

Part 7 — Exercises

Your teaching style is solve-first. Attempt these before reading any solutions.

1. **Rule of zero.** Delete the destructor, copy constructor, move constructor and `operator=` from `Tree`. What still compiles? What behaviour changes? Run the deep-chain test to see the change for yourself.
2. **Level-order iterator.** Write `LevelOrderIterator<IsConst>` using a `std::queue`, and `static_assert` that it's a forward iterator. Is it really multi-pass? What does copying one cost?
3. **Post-order iterator.** Harder: the iterator must store the `(node, nextChild)` frames between increments.
4. **`std::optional` API.** Change `root()` to return `std::optional<std::reference_wrapper<Node>>`. Compare the call sites with the `Node*` version. Which reads better?
5. **Cached subtree sizes.** Store `mSubtreeSize` in each `Node`, making `detach` O(k). What does `addChild` now cost, and which operations need updating?
6. **`insertBefore(sibling, value)`** at a given position. Keep the strong guarantee (look up `vector::insert`).
7. **Deducing `this` (C++23, `-std=c++23`).** Collapse a const/non-const pair into one function.
8. **Non-allocating `clear()`.** Destroy the tree in O(n) with O(1) extra memory, by repeatedly rotating a node's children onto a single chain.
9. **Binary search tree.** Build `dsa::Bst<T, Compare = std::less<T>>` following Part 6.2, with in-order iteration. Then compare its speed against `std::set` on sorted input.
10. **Pretty printer.** Write a non-member `printTree` that draws `|--` / `` `-- `` connectors like the comment at the top of `main`.

Appendix — Concept → where it is used

Concept	Used in
C1 headers / `#pragma once`	top of `Tree.hpp`
C2 namespaces	`namespace dsa`
C5 encapsulation, invariants	class comment; all data members private
C6 init lists, default member initializers, `explicit`	`Node` constructor, `mParent{nullptr}`, `mSize{0}`, `explicit Tree(T)`
C7 `= default` / `= delete`	`Tree() = default`, `Node(const Node&) = delete`
C8 const / noexcept / nodiscard / static	nearly every accessor; `depth`, `makeNode`, the traversal helpers
C9 nested classes, aliases	`Node`, `PreOrderIterator`, `size_type`, `iterator`
C10 `static_cast`	`erase`, the const traversal overloads
C11 friends, hidden friends	`friend class Tree`, `swap`, both `operator==`
C12 operator overloading	iterator operators, `operator==`, `operator=`
C13 RAII, destructor, stack limits	`~Tree`, `clear`
C14 `unique_ptr`, raw observers	`mRoot`, `mChildren`, `mParent`, `makeNode`
C15–C17 copy, move, copy-and-swap	copy/move constructors, `operator=`, `swap`
C18 parameter passing	`addChild` overloads, `Tree(T)`
C19–C22 templates, concepts, forwarding	whole class; `findIf`, `find`, `operator==`, `emplace*`
C23 vector and invalidation	children, work stacks, `reserve`-then-`push_back`
C24 queue, pair, structured bindings	`levelOrder`, copy constructor, `operator==`, `postOrder`
C25 algorithms	`detach` (`find_if`), `height` (`max`), tests (`accumulate`)
C26 lambdas and callables	traversal parameters, `children()`, `find`, `countNodes`, tests
C27 iterators	`PreOrderIterator`, `begin`/`end`
C28 views	`Node::children()`
C29 assert, static_assert	preconditions; the compile-time checks in `test.cpp`
C30 exceptions, guarantees	`emplaceRoot`, `attach`, `child(i)`; comments on each modifier
C31 error-reporting choices	`find` → `nullptr`, `emplaceRoot` → throw, handles → assert